

Lab 2: C++ Compilation, Debugging, Build Systems, and Version Control

Objective

This lab introduces the practical tools used in real-world C++ development: compiling code with GCC/G++, inspecting the compilation stages, debugging with GDB, organizing multi-file programs, automating builds with Makefiles, and tracking code changes with Git.

1 Part A: GCC/G++ Compilation Process

What to do

- Write a simple C++ program (e.g., a program that computes factorial or checks primality).
- Compile it using `g++` and run the executable.
- Explore each stage of compilation separately using compiler flags:
 - **Preprocessing** – generate the preprocessed output (`-E`)
 - **Compilation** – generate assembly code (`-S`)
 - **Assembly** – generate the object file (`-c`)
 - **Linking** – produce the final executable
- Note down the file produced at each stage and briefly describe what changed.
- Try compiling with different optimization levels (`-O0`, `-O2`) and compare.

2 Part B: Debugging Using GDB

What to do

- Take a C++ program with a deliberate bug (e.g., an off-by-one error or a segmentation fault from an out-of-bounds array access).
- Compile it with debugging symbols enabled (`-g`).
- Load the program in GDB and:
 - Set breakpoints at relevant functions/lines.
 - Step through the program line by line.
 - Inspect variable values at different points.
 - Use a watchpoint to track when a variable changes.
 - Identify and fix the bug.
- Record the sequence of GDB commands used and the final fix.

3 Part C: Multi-file Programs

What to do

- Split a single-file program into multiple files: separate header (`.h`) and source (`.cpp`) files for at least one class or module, plus a `main.cpp`.
- Use header guards (or `#pragma once`) correctly.
- Compile the multi-file program manually using `g++`, first generating object files for each source file and then linking them together.
- Explain why splitting code into multiple files is useful for larger projects.

4 Part D: Makefiles

What to do

- Write a `Makefile` for the multi-file program from Part C.
- The `Makefile` should include:
 - Rules to compile each source file into an object file.
 - A rule to link all object files into the final executable.
 - A `clean` target to remove generated files.
- Use variables (e.g., `CXX`, `CXXFLAGS`, `OBJS`) instead of hardcoding values.
- Run `make` and `make clean` to confirm the build works end-to-end.

5 Part E: Introduction to Git and Version Control

What to do

- Initialize a Git repository for the project from Parts C and D.
- Create a `.gitignore` file to exclude compiled binaries and object files.
- Make an initial commit, then:
 - Make a small code change and commit it separately.
 - View the commit history using `git log`.
 - Create a new branch, make a change on it, and merge it back into the main branch.
- Briefly note the purpose of `git status`, `git diff`, and `git checkout` based on your usage.

Submission Instructions

- Submit a **single Word document** (`.doc/.docx`) containing:
 1. Screenshots and outputs for each part (A–E).
 2. Source code for the multi-file program and the `Makefile`.
 3. Brief explanations/answers for each “What to do” item.
- **No other file formats (PDF, `.zip`, `.cpp`, etc.) will be accepted.**
- Ensure the document is well-organized with clear section headings matching Parts A–E.